Dv Guidance

Component Design Document

1 Description

Delta-V guidance. Produces the inertially fixed attitude reference for a delta-V burn. The burn frame has its third axis along the requested delta-V and its second axis as close as possible to a preferred direction. Both are parameters, set by the state manager when a burn is commanded, so the reference attitude is a constant between burn commands and the reference rate and acceleration are zero. The algorithm validates the frame before it is applied. A rotation about the burn axis, when wanted, is applied by a downstream reference rotation taking this reference as its base. Wraps the DvGuidanceAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 2
- **Number of Invokee Connectors** - 2
- **Number of Invoker Connectors** - *None*
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 2
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - *None*
- **Number of Data Dependencies** - *None*
- **Number of Packets** - *None*

3.2 Diagram

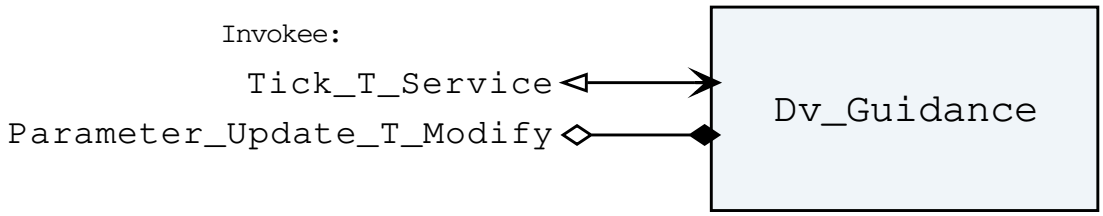


Figure 1: Dv Guidance component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Dv Guidance Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Service	service	Tick.T	Att_Ref.T	1
Parameter_ Update_T_Modify	modify	Parameter_ Update.T	-	1

Connector Descriptions:

- **Tick_T_Service** - Run the algorithm up to the current time and return the attitude reference it produces.
- **Parameter_Update_T_Modify** - The parameter update connector.

3.3.2 Invoker Connectors

None

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 2: Dv Guidance Set Id Bases Parameters

Name	Type
`Parameter_Id_Base`	`Parameter_Types.Parameter_Id_Base`

Parameter Descriptions:

- **`Parameter_Id_Base`** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains no data dependencies.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the delta-V guidance algorithm with the default parameter values, a stand-in burn frame which the first burn command replaces. The `init` subprogram takes no parameters.

3.6 Commands

The Dv Guidance component has no commands.

3.7 Parameters

Parameters for the Dv Guidance component. Together they define the burn frame, so they are set when a burn is commanded. The defaults form a valid frame and stand in until then.

Table 3: Dv Guidance Parameters

Local ID	Parameter Name	Type	Default Value
`0x0000 (0)`	`Requested_Dv_N`	`Packed_F32x3.T`	`[0.0,0.0,1.0]`
`0x0001 (1)`	`R_Hat_Pref_N`	`Packed_F32x3.T`	`[1.0,0.0,0.0]`

Parameter Descriptions:

- **`Requested_Dv_N`** - [m/s] Requested delta-V in inertial frame components. The third axis of the burn frame points along it. Must be finite and at least 1.0E-6 m/s long.
- **`R_Hat_Pref_N`** - [-] Preferred direction of the second axis of the burn frame in inertial frame components. Must be finite, within 1.0E-3 of unit length, and not within about 0.06 degrees of parallel or antiparallel to the requested delta-V. Only its component perpendicular to the delta-V shapes the frame.

3.8 Events

The Dv Guidance component has no events.

3.9 Data Products

The Dv Guidance component has no data products.

3.10 Data Dependencies

The Dv Guidance component has no data dependencies.

3.11 Packets

The Dv Guidance component has no packets.

3.12 Faults

The Dv Guidance component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Dv_Guidance_Tests* Test Suite

This is a unit test suite for the Dv Guidance component

Test Descriptions:

- **`Test`** - Check the reference returned for the burn frame of the algorithm's Python reference test, with zero reference rates, to ensure the Ada to C to C++ integration is sound, and that the stand-in frame serves before a burn frame is set.
- **`Test_Burn_Frame_Update`** - Ensure a newly set delta-V and preferred direction reach the reference on the next tick.
- **`Test_Invalid_Parameter`** - Ensure a staged burn frame the algorithm would reject is refused at validation and leaves the reference unchanged.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Att_Ref.T:

Table 4: Att_Ref Packed Record : 288 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sigma_Rn	Packed_F32x3.T	-	96	0	95
Omega_Rn_N	Packed_F32x3.T	-	96	96	191
Domega_Rn_N	Packed_F32x3.T	-	96	192	287

Field Descriptions:

- **Sigma_Rn** - *No description provided.*
- **Omega_Rn_N** - *No description provided.*
- **Domega_Rn_N** - *No description provided.*

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 5: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 6: Parameter Packed Record : 792 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_ Header.T	-	24	0	23	-
Buffer	Parameter_ Types. Parameter_ Buffer_Type	-	768	24	791	Header.Buffer_ Length

Field Descriptions:
- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 7: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_ Length_Type	0 to 96	8	16	23

Field Descriptions:
- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 8: Parameter_Update Packed Record : 824 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_Types.Parameter_Table_Id	0 to 65535	16	0	15	-
Operation	Parameter_Enums.Parameter_Operation_Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate 4 => Discard	8	16	23	-
Status	Parameter_Enums.Parameter_Update_Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter.T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 9: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces.Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces.Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 10: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 11: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.
Discard	4	Discard any parameters staged by this table that have not been updated.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 12: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.